



Topic Modeling Using Latent Dirichlet Allocation (LDA)

Last Updated : 23 Jul, 2025

In the era of information explosion, extracting meaningful insights from large collections of text data has become increasingly important. Topic modeling is a powerful technique for uncovering hidden themes or topics within a corpus of documents. Among the various methods available, Latent Dirichlet Allocation (LDA) stands out as one of the most popular and effective algorithms for topic modeling.

This article delves into what LDA is, the fundamentals of topic modeling, and its applications, and concludes with a summary of its significance.

What is Topic Modeling?

[Topic modeling](#) is a type of statistical modeling used to uncover the abstract topics that occur in a collection of documents. It is a form of unsupervised learning, which means it does not require labeled data. Instead, it relies on the co-occurrence patterns of words within the documents to discover latent topics.

Key Concepts in Topic Modeling

1. **Corpus:** A large collection of text documents.
2. **Topics:** Abstract themes or concepts that are represented by clusters of words that frequently appear together.
3. **Topic Distribution:** The distribution of topics within a document, indicating the presence and prominence of various topics.
4. **Word Distribution:** The distribution of words within a topic, highlighting the most representative words for each topic.

Importance of Topic Modeling

Topic modeling is crucial for several reasons:

- **Dimensionality Reduction:** It reduces the complexity of text data by representing documents in terms of a limited number of topics.
- **Information Retrieval:** It enhances the retrieval of relevant information by identifying underlying themes.
- **Data Exploration:** It provides a way to explore and understand large collections of text data by summarizing them into interpretable topics.

What is Latent Dirichlet Allocation (LDA)?

[Latent Dirichlet Allocation \(LDA\)](#) is a generative probabilistic model designed to discover latent topics in large collections of text documents. Introduced by David Blei, Andrew Ng, and Michael Jordan in 2003, LDA assumes that each document is a mixture of topics and that each topic is a mixture of

words. The goal of LDA is to identify these topics and determine the distribution of topics within each document and the distribution of words within each topic.

How LDA Works

LDA operates on the following principles:

1. **Document-Topic Distribution:** Each document is represented as a distribution over topics. This means that a document can belong to multiple topics, each with a certain probability.
2. **Topic-Word Distribution:** Each topic is represented as a distribution over words. This implies that a topic is defined by a set of words, each associated with a probability of appearing in that topic.
3. **Generative Process:** LDA uses a generative process to create documents. This involves:
 - Choosing a distribution of topics for each document.
 - For each word in the document, selecting a topic based on the topic distribution.
 - Generating the word from the selected topic's word distribution.

Implementing Modeling using Latent Dirichlet Allocation

Step 1: Install Necessary Libraries

This step involves installing the required libraries for text processing and topic modeling, including pandas, gensim, spacy, nltk, and matplotlib.

```
!pip install pandas gensim spacy nltk matplotlib
```

Step 2: Create and Save Sample Dataset

In this step, we create a sample dataset containing a text column and save it to a CSV file. The sample dataset consists of a list of 10 text entries, each containing a short sentence.

```
import pandas as pd

# Create a sample dataset
data = {
    'text_column': [
        'The cat sat on the mat.',
        'Dogs are great pets.',
        'I love to play football.',
        'Data science is an interdisciplinary field.',
        'Python is a great programming language.',
        'Machine learning is a subset of artificial intelligence.',
        'Artificial intelligence and machine learning are popular topics.',
        'Deep learning is a type of machine learning.',
        'Natural language processing involves analyzing text data.',
        'I enjoy hiking and outdoor activities.'
    ]
}
```

```
# Convert to DataFrame
df = pd.DataFrame(data)

# Save DataFrame to CSV
df.to_csv('sample_dataset.csv', index=False)
```

Step 3: Load Dataset

Load the sample dataset from the CSV file into a DataFrame.

```
import pandas as pd

# Load data
data = pd.read_csv('sample_dataset.csv')
```

Step 4: Preprocess Text Data

This step involves cleaning the text data by removing extra spaces, emails, apostrophes, and non-alphabet characters, and converting the text to lowercase.

```
import re

# Preprocess the text data
def preprocess_text(text):
    text = re.sub('\s+', ' ', text) # Remove extra spaces
    text = re.sub('\S*@\S*\s?', '', text) # Remove emails
    text = re.sub("'", '', text) # Remove apostrophes
    text = re.sub('[^a-zA-Z]', ' ', text) # Remove non-alphabet characters
    text = text.lower() # Convert to lowercase
    return text

data['cleaned_text'] = data['text_column'].apply(preprocess_text)
```

Step 5: Tokenize and Remove Stopwords

Tokenize the cleaned text data and remove stopwords using NLTK.

```
import gensim
import nltk
from nltk.corpus import stopwords

# Download NLTK stopwords
nltk.download('stopwords')
stop_words = stopwords.words('english')

# Tokenize and remove stopwords
def tokenize(text):
    tokens = gensim.utils.simple_preprocess(text, deacc=True)
```

```
tokens = [token for token in tokens if token not in stop_words]
return tokens

data['tokens'] = data['cleaned_text'].apply(tokenize)
```

Step 6: Lemmatize Tokens

Lemmatize the tokens using spaCy.

```
import spacy

# Load spaCy model
nlp = spacy.load('en_core_web_sm', disable=['parser', 'ner'])

def lemmatize(tokens):
    doc = nlp(" ".join(tokens))
    return [token.lemma_ for token in doc]

data['lemmas'] = data['tokens'].apply(lemmatize)
```

Step 7: Create Dictionary and Corpus

Create a dictionary and corpus from the lemmatized tokens.

```
import gensim.corpora as corpora

# Create dictionary and corpus
id2word = corpora.Dictionary(data['lemmas'])
texts = data['lemmas']
corpus = [id2word.doc2bow(text) for text in texts]
```

Step 8: Build LDA Model

Build an LDA model with the specified number of topics.

```
# Build LDA model
lda_model = gensim.models.ldamodel.LdaModel(corpus=corpus,
                                             id2word=id2word,
                                             num_topics=3,
                                             random_state=100,
                                             update_every=1,
                                             chunksize=100,
                                             passes=10,
                                             alpha='auto',
                                             per_word_topics=True)
```

Step 9: Print Topics

Print the topics generated by the LDA model.

```
# Print the topics
topics = lda_model.print_topics(num_words=10)
for topic in topics:
    print(topic)
```

Step 10: Compute Coherence Score

Compute the coherence score to evaluate the quality of the topics.

```
from gensim.models import CoherenceModel

# Compute coherence score
coherence_model_lda = CoherenceModel(model=lda_model, texts=data['lemmas'],
dictionary=id2word, coherence='c_v')
coherence_lda = coherence_model_lda.get_coherence()
print('\nCoherence Score: ', coherence_lda)
```

Complete implementation of Topic Modeling using LDA

```
import pandas as pd

# Create a sample dataset
data = {
    'text_column': [
        'The cat sat on the mat.',
        'Dogs are great pets.',
        'I love to play football.',
        'Data science is an interdisciplinary field.',
        'Python is a great programming language.',
        'Machine learning is a subset of artificial intelligence.',
        'Artificial intelligence and machine learning are popular topics.',
        'Deep learning is a type of machine learning.',
        'Natural language processing involves analyzing text data.',
        'I enjoy hiking and outdoor activities.'
    ]
}

# Convert to DataFrame
df = pd.DataFrame(data)

# Save DataFrame to CSV
df.to_csv('sample_dataset.csv', index=False)

import pandas as pd
import gensim
import gensim.corpora as corpora
from gensim.models import CoherenceModel
import spacy
import nltk
from nltk.corpus import stopwords
import re
import matplotlib.pyplot as plt

# Download NLTK stopwords
nltk.download('stopwords')
stop_words = stopwords.words('english')

# Load data
data = pd.read_csv('sample_dataset.csv') # Load the sample dataset
```

```

# Preprocess the text data
def preprocess_text(text):
    text = re.sub('\s+', ' ', text) # Remove extra spaces
    text = re.sub('\S*@\S*\s?', '', text) # Remove emails
    text = re.sub("'", '', text) # Remove apostrophes
    text = re.sub('[^a-zA-Z]', ' ', text) # Remove non-alphabet characters
    text = text.lower() # Convert to lowercase
    return text

data['cleaned_text'] = data['text_column'].apply(preprocess_text) # Replace 'text_column' with your column name

# Tokenize and remove stopwords
def tokenize(text):
    tokens = gensim.utils.simple_preprocess(text, deacc=True)
    tokens = [token for token in tokens if token not in stop_words]
    return tokens

data['tokens'] = data['cleaned_text'].apply(tokenize)

# Lemmatization using spaCy
nlp = spacy.load('en_core_web_sm', disable=['parser', 'ner'])
def lemmatize(tokens):
    doc = nlp(" ".join(tokens))
    return [token.lemma_ for token in doc]

data['lemmas'] = data['tokens'].apply(lemmatize)

# Create dictionary and corpus
id2word = corpora.Dictionary(data['lemmas'])
texts = data['lemmas']
corpus = [id2word.doc2bow(text) for text in texts]

# Build LDA model
lda_model = gensim.models.ldamodel.LdaModel(corpus=corpus,
                                             id2word=id2word,
                                             num_topics=3,
                                             random_state=100,
                                             update_every=1,
                                             chunksize=100,
                                             passes=10,
                                             alpha='auto',
                                             per_word_topics=True)

# Print the topics
topics = lda_model.print_topics(num_words=10)
for topic in topics:
    print(topic)

# Compute coherence score
coherence_model_lda = CoherenceModel(model=lda_model, texts=data['lemmas'], dictionary=id2word,
coherence='c_v')
coherence_lda = coherence_model_lda.get_coherence()
print('\nCoherence Score: ', coherence_lda)

```

Output:

```

(0, '0.115*"learning" + 0.066*"type" + 0.066*"programming" + 0.066*"python" + 0.066*"deep" +
0.066*"great" + 0.065*"language" + 0.065*"machine" + 0.016*"datum" + 0.016*"love"')
(1, '0.062*"outdoor" + 0.062*"activity" + 0.062*"football" + 0.062*"enjoy" + 0.062*"cat" +
0.062*"play" + 0.062*"hike" + 0.062*"mat" + 0.062*"sit" + 0.062*"love"')
(2, '0.066*"machine" + 0.066*"datum" + 0.066*"artificial" + 0.066*"intelligence" +
0.038*"language" + 0.038*"great" + 0.038*"learning" + 0.038*"popular" + 0.038*"learn" +
0.038*"processing"')

```

Coherence Score: 0.5839748062472863

The output shows three topics, each represented by a list of words with associated weights, indicating the importance of each word in that topic. The coherence score, which is 0.5839748062472863,

measures the interpretability of the topics. Higher scores generally indicate more coherent and interpretable topics.

The coherence score of 0.5839748062472863 suggests that the topics are reasonably coherent and interpretable, although there might still be room for improvement. Coherence scores range from 0 to 1, with higher scores indicating better topic quality.

Applications of LDA and Topic Modeling

LDA and topic modeling have a wide range of applications across various domains. Here are some notable examples:

1. **Document Classification:** Topic modeling can be used to classify documents into categories based on their dominant topics. For instance, in news aggregation, articles can be categorized into topics such as politics, sports, technology, and entertainment.
2. **Recommendation Systems:** By understanding the topics of documents or products, recommendation systems can suggest relevant items to users. For example, an online bookstore can recommend books based on the topics a user has shown interest in.
3. **Content Summarization:** Topic modeling helps in summarizing large collections of documents by extracting the main themes. This is particularly useful in literature reviews and survey papers where summarizing vast amounts of research is necessary.
4. **Trend Analysis:** LDA can be employed to analyze trends over time by examining the distribution of topics in documents from different time periods. This is valuable for market research and understanding evolving interests.
5. **Sentiment Analysis:** Combining topic modeling with sentiment analysis can provide insights into the sentiment associated with specific topics. This is useful in opinion mining and social media analysis.

Conclusion

Latent Dirichlet Allocation (LDA) is a powerful tool for topic modeling, enabling the discovery of hidden themes within large collections of text documents. By representing documents as mixtures of topics and topics as mixtures of words, LDA provides a probabilistic framework for understanding and exploring text data. Its applications span numerous fields, from document classification and recommendation systems to trend analysis and sentiment analysis. Despite some limitations, such as the need for large datasets and computational resources, LDA remains a foundational technique in the realm of natural language processing and text mining.

Comment

S sujithc...

4

Article Tags:

[Blogathon](#)

[NLP](#)

[AI-ML-DS](#)

[AI-ML-DS With Python](#)

[+1 More](#)

Explore

[Introduction to NLP](#)

[Libraries for NLP](#)

Text Normalization in NLP

Text Representation and Embedding Techniques

NLP Deep Learning Techniques

NLP Projects and Practice



Corporate & Communications Address:
A-143, 7th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)

Registered Address:
K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305

Company

About Us
Legal
Privacy
Policy
Contact Us
Advertise with us
GFG
Corporate Solution

Explore

POTD
Job-A-Thon
Blogs
Nation
Skill Up

Tutorials

Programming Languages
DSA
Web
Technology
AI, ML & Data Science
DevOps
CS Core Subjects
Interview Preparation
Software and Tools

Courses

ML and Data Science
DSA and Placements
Web Development
Programming Languages
DevOps & Cloud
GATE
Trending Technologies

Videos

DSA
Python
Java
C++
Web Development
Data Science
CS Subjects

Preparation Corner

Interview Corner
Aptitude
Puzzles
GfG 160
System Design



@GeeksforGeeks, Sanchhaya Education Private Limited, All rights reserved

Do Not Sell or Share My Personal Information